UNIVERSIDADE CATÓLICA DE BRASÍLIA
PRÓ-REITORIA ACADÊMICA – PROACAD
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ADAM AMARILDO GUEDES CARNEIRO
MIGUEL OLIVEIRA SILVA
YUDI YUMETSU UEYAMA

FLUXO DE CONTROLE EM ASSEMBLY RISC-V:
desvios, saltos condicionais, pilha de execução e recursão

BRASÍLIA
2026

ADAM AMARILDO GUEDES CARNEIRO
MIGUEL OLIVEIRA SILVA
YUDI YUMETSU UEYAMA

FLUXO DE CONTROLE EM ASSEMBLY RISC-V:
desvios, saltos condicionais, pilha de execução e recursão

Relatório técnico-acadêmico apresentado à unidade curricular Arquitetura de Computadores do curso de Ciência da Computação da Universidade Católica de Brasília, como requisito parcial de avaliação da atividade teórico-prática do Tema 4 – Fluxo de Controle em Assembly.

Professor: Weldes Lima Oliveira.

BRASÍLIA
2026

RESUMO

Neste trabalho, procuramos entender o que acontece dentro do processador quando um programa executa um laço, toma uma decisão ou chama uma função, inclusive quando essa função chama a si mesma. Usamos a arquitetura RISC-V de 32 bits (RV32I, com a extensão M para multiplicação) e o simulador RARS. Como o simulador não informa ciclos de *clock*, escrevemos também um pequeno rastreador em Python que executa o mesmo código e registra cada instrução. A prova de conceito lê um número digitado pelo usuário, mostrando uma mensagem de erro e repetindo a leitura enquanto ele for negativo, e depois calcula o fatorial desse número de forma recursiva, com um quadro de pilha de 8 bytes por chamada. Para comparação, um segundo programa calcula o mesmo fatorial de forma iterativa. Ao longo do relatório, mostramos como o contador de programa (PC) é atualizado em cada desvio, como as instruções de desvio são codificadas em binário e como a pilha cresce e diminui durante a recursão. Também reproduzimos de propósito alguns erros comuns, como o laço que não termina por causa de uma condição invertida e o retorno perdido quando o registrador ra não é salvo. Nas medições, o fatorial recursivo executou $13n - 4$ instruções e fez $4n - 1$ acessos à memória, enquanto o iterativo executou $4n + 1$ instruções sem acessar a memória de dados. Para $n = 5$, isso representa cerca de 97 ciclos contra 37 em um *pipeline* de cinco estágios. A principal conclusão é que *if*, *while* e chamadas de função têm um custo real no hardware, e que entender esse custo ajuda tanto a escrever código mais rápido quanto a encontrar erros difíceis.

Palavras-chave: Arquitetura de computadores. RISC-V. Linguagem de montagem. Fluxo de controle. Pilha de execução.

LISTA DE FIGURAS

Figura 1 – Mapa de memória do RARS com os rótulos da prova de conceito 16

Figura 2 – Caminho de dados simplificado do RV32I para a seleção do próximo PC 17

Figura 3 – Fluxograma do laço while (main) e da decisão if/else com recursão (fatorial) 19

Figura 4 – Codificação das instruções bge (formato B) e jal (formato J) da função fatorial.....20

Figura 5 – Conteúdo da pilha no caso base de fatorial(5) 21

Figura 6 – Execução completa de fatorial_completo_tema4.s no RARS 1.6 com as entradas −3, −1 e 5: console, total de instruções e registradores 28

Figura 7 – Pilha no caso base de fatorial(5) lida da memória do RARS 1.6 (execução interrompida no 44º passo) 28

Figura 8 – Instruções executadas e acessos à memória de dados em função de n 31

LISTA DE QUADROS

Quadro 1 – Registradores inteiros do RV32I e convenção de chamada 12
Quadro 2 – Instruções de controle de fluxo do RV32I e pseudo-instruções usadas na PoC 13
Quadro 3 – Correspondência entre flags do x86 e mecanismos do RISC-V 14
Quadro 4 – Fonte do próximo PC e registrador escrito por classe de instrução 17
Quadro 5 – Padrões de tradução de estruturas de controle de C para RISC-V 18
Quadro 6 – Ambiente de desenvolvimento, simulação e medição 22
Quadro 7 – Falhas reproduzidas, sintomas observados e correções 27

LISTA DE TABELAS

Tabela 1 – Rastro das três passagens pelo laço while (entradas −3, −1 e 5).....24
Tabela 2 – Estado dos registradores nos eventos de chamada e retorno de fatorial(3) 26
Tabela 3 – Resultados esperados e obtidos na execução dos programas.....29
Tabela 4 – Instruções executadas por classe e ciclos no modelo multiciclo.....30
Tabela 5 – Custo do fatorial recursivo e iterativo em função de n 31
Tabela 6 – Rastro instrução a instrução de fatorial(3) 41

LISTA DE ABREVIATURAS E SIGLAS

ABI	*Application Binary Interface* (interface binária de aplicação)
CISC	*Complex Instruction Set Computer*
CPI	Ciclos por instrução
EFLAGS	Registrador de sinalizadores da arquitetura x86
fp	*Frame pointer* (ponteiro de quadro; s0/x8 no RISC-V)
ISA	*Instruction Set Architecture* (arquitetura do conjunto de instruções)
PC	*Program Counter* (contador de programa)
psABI	*Processor-specific ABI* do RISC-V
ra	*Return address* (endereço de retorno; x1)
RARS	*RISC-V Assembler and Runtime Simulator*
RAW	*Read-After-Write* (dependência de dados verdadeira)
RISC	*Reduced Instruction Set Computer*
sp	*Stack pointer* (ponteiro de pilha; x2)
ULA	Unidade lógica e aritmética

SUMÁRIO

1 INTRODUÇÃO 9
2 FUNDAMENTAÇÃO TEÓRICA 11
2.1 CICLO DE INSTRUÇÃO E CONTADOR DE PROGRAMA 11
2.2 A ARQUITETURA RV32I: REGISTRADORES E FORMATOS 11
2.3 DESVIOS CONDICIONAIS E SALTOS INCONDICIONAIS 12
2.4 REGISTRADORES DE ESTADO: O MODELO X86 E O MODELO RISC-V 13
2.5 PROCEDIMENTOS, CONVENÇÃO DE CHAMADA E PILHA DE EXECUÇÃO 14
2.6 RECURSÃO, ITERAÇÃO E DESEMPENHO 15
3 MAPEAMENTO ARQUITETURAL 16
3.1 ORGANIZAÇÃO DA MEMÓRIA NO SIMULADOR 16
3.2 CAMINHO DE DADOS DA ATUALIZAÇÃO DO PC 16
3.3 DAS ESTRUTURAS DE ALTO NÍVEL AO ASSEMBLY 18
3.4 CODIFICAÇÃO BINÁRIA DAS INSTRUÇÕES DE DESVIO 19
3.5 QUADRO DE PILHA DA FUNÇÃO FATORIAL 20
4 METODOLOGIA E DESENVOLVIMENTO 22
4.1 AMBIENTE DE SIMULAÇÃO E MEDIÇÃO 22
4.2 ESTRUTURA DA PROVA DE CONCEITO 23
4.3 PARTE 1: LAÇO WHILE 23
4.4 PARTES 2 E 3: IF/ELSE, RECURSÃO E A VERSÃO ITERATIVA 25
4.5 ESTOURO SILENCIOSO DE 32 BITS 26
4.6 REPRODUÇÃO DOS DESAFIOS COMUNS 27
4.7 PROCEDIMENTO DE EXECUÇÃO NO RARS 27
5 DISCUSSÃO DOS RESULTADOS 29
5.1 VALIDAÇÃO FUNCIONAL 29
5.2 PERFIL DE INSTRUÇÕES DO PROGRAMA 29
5.3 RECURSÃO VERSUS ITERAÇÃO 30
5.4 DESVIOS, PIPELINE E PREVISÃO DE DESVIOS 32
5.5 ESTOURO ARITMÉTICO E AUSÊNCIA DE FLAGS 32
5.6 FALHAS DE CONTROLE E SEGURANÇA 33
5.7 ALINHAMENTO DO QUADRO DE PILHA E DOS DADOS 33
5.8 LIMITAÇÕES 34
5.9 IMPACTOS NO DESENVOLVIMENTO DE SOFTWARE DE ALTO DESEMPENHO 34
6 CONCLUSÃO 35
REFERÊNCIAS 36

APÊNDICE A – CÓDIGO-FONTE DE FATORIAL_COMPLETO_TEMA4.S 37
APÊNDICE B – CÓDIGO-FONTE DE FATORIAL_ITERATIVO.ASM 39
APÊNDICE C – CÓDIGO-FONTE DE ERROS_COMUNS.ASM 40
APÊNDICE D – RASTRO COMPLETO DE FATORIAL(3).....41
APÊNDICE E – ROTEIRO DA DEMONSTRAÇÃO PRÁTICA (POC) 42

1 INTRODUÇÃO

Quem programa em C, Java ou Python escreve *if*, *while* e funções sem pensar muito em como isso funciona por baixo. O processador, porém, não conhece nenhuma dessas estruturas. Ele busca uma instrução na memória, executa e passa para a próxima, e o endereço dessa próxima instrução fica guardado em um único registrador, o contador de programa (PC). Toda decisão que um programa toma acaba se resumindo a isso: o PC avança para a instrução seguinte ou recebe outro endereço (PATTERSON; HENNESSY, 2017; TANENBAUM; AUSTIN, 2013).

Escolhemos estudar esse mecanismo porque ele aparece em praticamente qualquer programa e tem consequências que vão além da teoria. Em processadores com *pipeline*, um desvio pode obrigar o processador a descartar instruções que ele já tinha começado a buscar, e por isso há décadas se pesquisam formas de prever desvios (SMITH, 1981; HENNESSY; PATTERSON, 2019). A pilha de execução, usada nas chamadas de função, também é fonte de problemas conhecidos, como o estouro de pilha e a corrupção do endereço de retorno, que já foi explorada em ataques de *buffer overflow* (ALEPH ONE, 1996).

Este relatório corresponde ao Tema 4 da atividade avaliativa (Fluxo de Controle em Assembly: desvios, saltos condicionais e manipulação de dados). Nosso objetivo foi explicar, com dados medidos em simulação, como as estruturas de controle de alto nível viram instruções de máquina. Mais especificamente, buscamos:

- entender o papel do PC, dos desvios condicionais e dos saltos no ciclo de instrução;
- comparar o uso de *flags* no x86 com a comparação feita dentro da própria instrução de desvio no RISC-V;
- escrever em Assembly um laço *while*, uma decisão *if/else* e uma função recursiva (o fatorial), acompanhando a pilha a cada chamada;
- medir quanto custa a versão recursiva em relação à iterativa;
- provocar de propósito os erros mais comuns do tema e observar o que o processador faz em cada caso.

Usamos o RISC-V de 32 bits por ser uma arquitetura aberta e bem mais enxuta que a x86, o que facilita enxergar o mecanismo sem os detalhes de compatibilidade histórica das arquiteturas comerciais (ASANOVIĆ; PATTERSON, 2014; WATERMAN, 2016). Os programas rodaram no RARS, um simulador derivado do MARS e bastante

usado no ensino de Assembly (VOLLMAR; SANDERSON, 2006). O código da prova de conceito partiu do roteiro de apresentação montado pelo grupo, e todas as medições foram feitas sobre exatamente esse código, para que os *slides*, a demonstração e o relatório falassem da mesma coisa.

Na seção 2 apresentamos a base teórica. A seção 3 traz os diagramas da arquitetura, desenhados com os endereços reais do programa. A seção 4 descreve o ambiente e o desenvolvimento da prova de conceito, e a seção 5 discute os resultados. As conclusões estão na seção 6, e os apêndices reúnem o código completo, o rastro de execução e o roteiro da demonstração.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 CICLO DE INSTRUÇÃO E CONTADOR DE PROGRAMA

Na organização de von Neumann, instruções e dados ficam na mesma memória, e o processador repete sem parar um ciclo de busca, decodificação e execução (STALLINGS, 2017). Primeiro, o endereço guardado no PC é enviado à memória e a instrução correspondente é lida. Em seguida, a unidade de controle interpreta o código de operação (*opcode*) e os demais campos da instrução. Por fim, a ULA realiza a operação, e o resultado vai para um registrador ou para a memória.

Para este trabalho, o passo que mais interessa é o último de cada ciclo: a atualização do PC. Quando nada interfere, ele simplesmente avança para a próxima instrução. No RISC-V, em que as instruções básicas têm 32 bits, isso equivale a PC ← PC + 4. Só as instruções de controle de fluxo colocam no PC um valor diferente, e é apenas por meio delas que um programa consegue decidir, repetir ou chamar uma função (PATTERSON; HENNESSY, 2017).

2.2 A ARQUITETURA RV32I: REGISTRADORES E FORMATOS

O RV32I é o conjunto básico de instruções inteiras de 32 bits do RISC-V. Ele tem 32 registradores de uso geral, de x0 a x31, todos com 32 bits. O x0 vale sempre zero, e o PC fica à parte, sem poder ser acessado como os outros (WATERMAN; ASANOVIĆ, 2019). Usamos também a extensão M, que acrescenta multiplicação e divisão, porque o fatorial precisa da instrução `mul`. Outra característica importante é que o RISC-V segue o modelo *load/store*: só `lw` e `sw` acessam a memória de dados, e as contas são sempre feitas entre registradores.

Cada registrador tem ainda um nome simbólico, definido pela convenção de chamada (ABI) e mostrado no Quadro 1. Para o hardware, com exceção do x0, todos são iguais. A convenção é um acordo de software, mas é ela que permite que funções escritas e compiladas separadamente funcionem juntas (PATTERSON; WATERMAN, 2017).

Quadro 1 – Registradores inteiros do RV32I e convenção de chamada

Registrador	Nome ABI	Uso	Preservado na chamada?
x0	`zero`	Constante zero (escritas são descartadas)	—
x1	`ra`	Endereço de retorno (*return address*)	Não (chamador salva)
x2	`sp`	Ponteiro de pilha (*stack pointer*)	Sim
x3 / x4	`gp` / `tp`	Ponteiro global / ponteiro de *thread*	—
x5–x7, x28–x31	`t0`–`t6`	Temporários	Não
x8	`s0` / `fp`	Registrador salvo / ponteiro de quadro (*frame pointer*)	Sim (chamado salva)
x9, x18–x27	`s1`–`s11`	Registradores salvos	Sim (chamado salva)
x10–x11	`a0`–`a1`	Argumentos e valores de retorno	Não
x12–x17	`a2`–`a7`	Argumentos (a7 também seleciona o serviço da `ecall`)	Não

Fonte: Adaptado de Waterman e Asanović (2019) e Patterson e Waterman (2017).

Todas as instruções do RV32I ocupam 32 bits e seguem seis formatos: R, I, S, B, U e J. O *opcode* fica sempre nos bits 6 a 0, e os campos dos registradores (`rs1`, `rs2` e `rd`) estão sempre na mesma posição. Isso permite ler os registradores antes mesmo de terminar a decodificação, e Waterman (2016) aponta essa regularidade como uma escolha pensada para facilitar implementações rápidas. Os formatos que mais interessam aqui são o B, dos desvios condicionais, e o J, do salto com ligação, detalhados na seção 3.4.

2.3 DESVIOS CONDICIONAIS E SALTOS INCONDICIONAIS

Um desvio condicional compara dois registradores. Se a condição for verdadeira, ele soma um deslocamento ao PC; se não for, o PC segue normalmente para PC + 4. No primeiro caso dizemos que o desvio foi *tomado* e, no segundo, que *não foi tomado*. Já o salto incondicional muda o PC sempre. O RISC-V tem seis desvios condicionais e dois saltos, e as outras formas que aparecem no nosso código, como `ble`, `j` e `jr`, são pseudo-instruções que o montador traduz para essas oito (Quadro 2).

Quadro 2 – Instruções de controle de fluxo do RV32I e pseudo-instruções usadas na PoC

Instrução	Formato	Semântica	Observação
`beq rs1, rs2, L`	B	se R[rs1] = R[rs2]: PC ← PC + imm	igualdade
`bne rs1, rs2, L`	B	se R[rs1] ≠ R[rs2]: PC ← PC + imm	desigualdade
`blt rs1, rs2, L`	B	se R[rs1] < R[rs2] (com sinal)	complemento de dois
`bge rs1, rs2, L`	B	se R[rs1] ≥ R[rs2] (com sinal)	complemento de dois
`bltu` / `bgeu`	B	comparações sem sinal	equivalem ao uso do *carry*
`jal rd, L`	J	R[rd] ← PC + 4; PC ← PC + imm	chamada de procedimento
`jalr rd, imm(rs1)`	I	R[rd] ← PC + 4; PC ← (R[rs1] + imm) & ~1	retorno e saltos indiretos
`bgt a, b, L` / `ble a, b, L`	pseudo	`blt b, a, L` / `bge b, a, L`	operandos trocados
`j L`	pseudo	`jal x0, L`	salto sem ligação
`ret` / `jr ra`	pseudo	`jalr x0, 0(ra)`	retorno de procedimento
`mv`, `li`, `la`	pseudo	`add`/`addi`; `addi` ou `lui+addi`; `auipc+addi`	não alteram o fluxo

Fonte: Adaptado de Waterman e Asanović (2019).

O deslocamento é contado a partir do próprio PC e é sempre par, já que o bit menos significativo não chega a ser armazenado. No formato B ele alcança 4 KiB para frente ou para trás; no formato J, 1 MiB. Para saltos mais longos, usa-se a combinação de `auipc` com `jalr` (WATERMAN; ASANOVIĆ, 2019).

2.4 REGISTRADORES DE ESTADO: O MODELO X86 E O MODELO RISC-V

Na arquitetura x86, o desvio acontece em duas etapas. Uma instrução aritmética ou de comparação (`cmp`) atualiza o registrador de sinalizadores, o EFLAGS, ligando ou desligando bits como Zero (ZF), *Carry* (CF), Sinal (SF) e *Overflow* (OF). Depois, um salto condicional como `jz`, `jl` ou `jge` consulta esses bits e decide se o PC, que lá se chama EIP ou RIP, vai mudar (STALLINGS, 2017; TANENBAUM; AUSTIN, 2013).

O RISC-V simplesmente não tem registrador de *flags*. A comparação e o desvio acontecem na mesma instrução, que lê os dois registradores e já decide o caminho. O manual da arquitetura explica que, dessa forma, nenhuma instrução aritmética precisa atualizar um estado escondido, o que simplifica processadores com *pipeline* e com execução fora de ordem, nos quais as *flags* criam dependências entre instruções

que não têm relação nenhuma entre si (WATERMAN; ASANOVIĆ, 2019; WATERMAN, 2016). O Quadro 3 mostra o que corresponde a cada *flag* no RISC-V.

Quadro 3 – Correspondência entre flags do x86 e mecanismos do RISC-V

Flag (x86)	Significado	Equivalente no RISC-V
ZF (Zero)	resultado igual a zero	`beq`/`bne` comparando registradores (ou com `zero`)
SF (Sinal)	resultado negativo	`blt`/`bge` com sinal, p. ex. `blt t1, zero, L`
CF (*Carry*)	vai-um / empréstimo sem sinal	`bltu`/`bgeu` ou `sltu` explícito
OF (*Overflow*)	estouro em complemento de dois	não há: verificação explícita em software (seção 5.5)

Fonte: Elaborado pelos autores com base em Stallings (2017) e Waterman e Asanović (2019).

Um exemplo ajuda a visualizar. A condição `if (a < b)` vira `cmp eax, ebx` seguido de `jge senao` no x86, enquanto no RISC-V basta `bge a0, a1, senao`. Repare que, nos dois casos, o desvio testa a condição contrária à do código em C, porque ele serve justamente para pular o bloco quando a condição é falsa. Essa inversão confunde bastante quem escreve Assembly à mão e volta a aparecer na seção 3.3.

2.5 PROCEDIMENTOS, CONVENÇÃO DE CHAMADA E PILHA DE EXECUÇÃO

Patterson e Hennessy (2017) dividem a execução de um procedimento em seis passos: colocar os argumentos em um lugar acessível, transferir o controle, reservar a memória necessária, executar a tarefa, deixar o resultado onde o chamador possa pegá-lo e voltar ao ponto de origem. No RISC-V, argumentos e resultado passam pelos registradores `a0` a `a7`. A transferência é feita por `jal ra, rotulo`, que guarda em `ra` o endereço de retorno (PC + 4) antes de saltar, e a volta é feita por `ret` ou `jr ra`, ambos equivalentes a `jalr x0, 0(ra)`.

A memória do terceiro passo vem da pilha de execução, uma região que cresce em direção aos endereços menores e cujo topo é apontado por `sp`. Cada chamada que precisa guardar dados reserva seu próprio quadro de pilha (*stack frame*), subtraindo um valor de `sp` no início da função, o prólogo, e devolvendo esse espaço no final, o epílogo. A convenção oficial do RISC-V pede que `sp` fique alinhado em 16 bytes (WATERMAN; ASANOVIĆ, 2019). Uma função que não chama nenhuma outra (função folha) pode até dispensar a pilha. Já uma função que chama outra precisa salvar pelo menos `ra`, porque o `jal` interno vai escrever por cima dele.

A convenção também separa os registradores em dois grupos. Os temporários e os de argumento (`t0–t6`, `a0–a7`), assim como o `ra`, podem ser destruídos pela função chamada, e quem chama é que deve guardá-los se precisar deles depois. Os registradores salvos (`s0–s11`) e o `sp` precisam sair da função com o mesmo valor com que entraram. O `s0` ainda pode servir como ponteiro de quadro (*frame pointer*), uma referência fixa ao quadro atual mesmo quando `sp` muda (PATTERSON; WATERMAN, 2017).

2.6 RECURSÃO, ITERAÇÃO E DESEMPENHO

Uma função recursiva é uma função que chama a si mesma. Como cada chamada tem seus próprios argumentos e seu próprio endereço de retorno, cada nível da recursão precisa de um quadro de pilha separado, e a pilha cresce na mesma proporção que a profundidade da recursão. Se o caso base nunca for alcançado, ela cresce até acabar (*stack overflow*). A versão iterativa do mesmo algoritmo guarda o estado em registradores e reaproveita as mesmas variáveis a cada volta. Em alguns casos, compiladores otimizadores conseguem transformar uma recursão em laço, principalmente quando a chamada recursiva é a última coisa que a função faz, a chamada recursão de cauda (AHO et al., 2008).

Para comparar as duas versões, usamos a equação clássica de desempenho: tempo de CPU = número de instruções × ciclos por instrução (CPI) × período do *clock* (HENNESSY; PATTERSON, 2019). O fluxo de controle mexe nos dois primeiros termos. Chamadas e retornos acrescentam instruções de prólogo, epílogo, `jal` e `jr`. Além disso, em um *pipeline*, cada desvio tomado pode obrigar o processador a descartar as instruções que já tinha buscado (risco de controle), e uma instrução que usa um valor recém-carregado por `lw` precisa esperar um ciclo (risco de dados do tipo RAW, ou *load-use*). Para diminuir os riscos de controle, os processadores usam previsores de desvio, que aproveitam o fato de um mesmo desvio costumar se comportar do mesmo jeito ao longo da execução (SMITH, 1981; HENNESSY; PATTERSON, 2019).

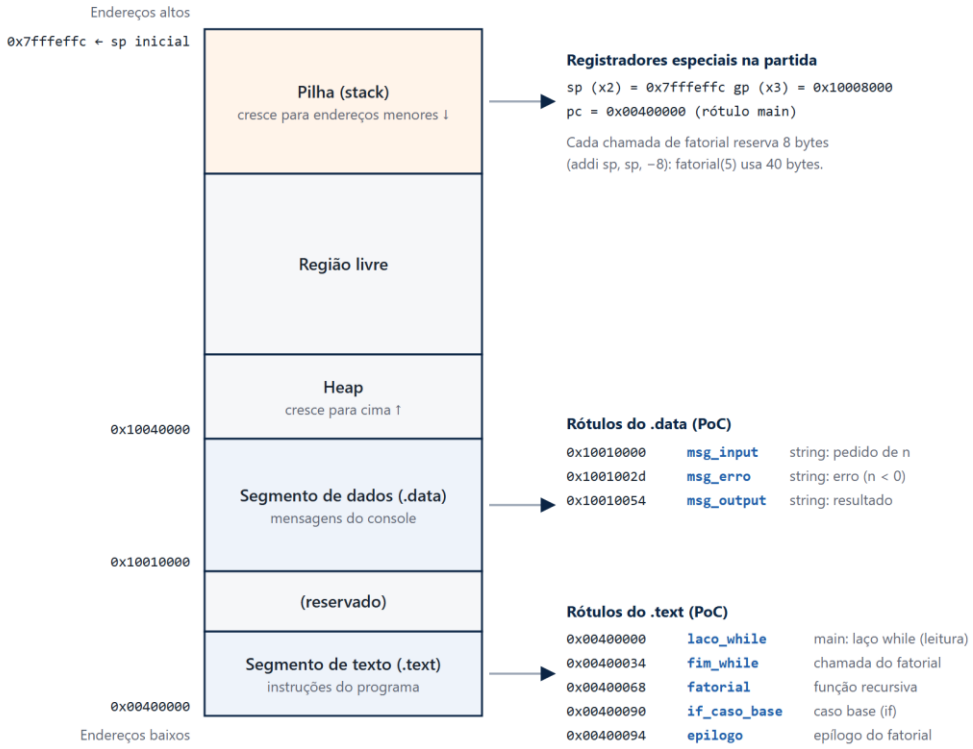
3 MAPEAMENTO ARQUITETURAL

Todos os diagramas desta seção foram feitos a partir do próprio programa da prova de conceito. Os endereços, os códigos binários e o conteúdo da memória que aparecem neles são os valores obtidos na montagem e na execução descritas na seção 4, e os nomes dos registradores seguem o RV32I.

3.1 ORGANIZAÇÃO DA MEMÓRIA NO SIMULADOR

A Figura 1 mostra como o RARS organiza a memória na configuração padrão. As instruções ficam no segmento de texto, que começa em `0x00400000`. Os dados estáticos começam em `0x10010000`, e a pilha parte de `0x7fffeffc` e cresce para baixo. No nosso programa, o laço `laco_while` está logo no início do segmento de texto, a função `fatorial` começa em `0x00400068` e o epílogo dela fica em `0x00400094`.

Figura 1 – Mapa de memória do RARS com os rótulos da prova de conceito

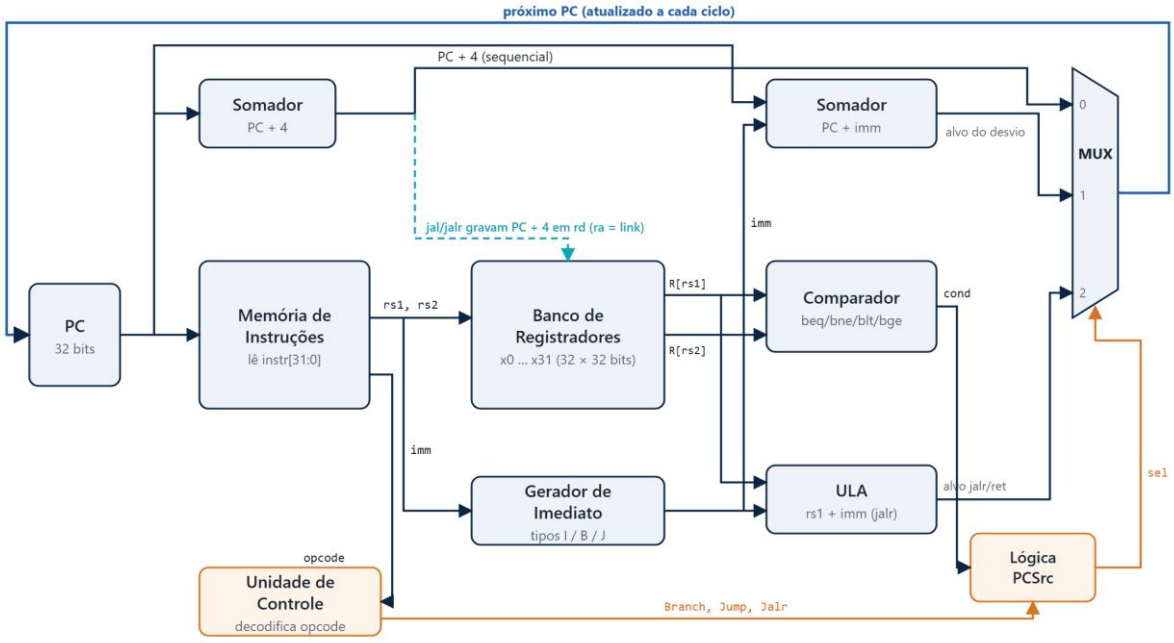


3.2 CAMINHO DE DADOS DA ATUALIZAÇÃO DO PC

A Figura 2 mostra apenas a parte de um caminho de dados monociclo do RV32I que decide qual será o próximo PC. São três possibilidades, escolhidas por um

multiplexador. A primeira é PC + 4, o caminho normal. A segunda é PC + imediato, calculado por um somador próprio e usado nos desvios tomados e no `jal`. A terceira é R[rs1] + imediato, calculado pela ULA e usado pelo `jalr`, que é como o `jr ra` acaba sendo executado. Quem escolhe entre elas é a lógica PCSrc, a partir dos sinais da unidade de controle e do resultado do comparador. O `jal` e o `jalr` ainda gravam PC + 4 no registrador de destino, e é essa ligação (*link*) que permite voltar depois.

Figura 2 – Caminho de dados simplificado do RV32I para a seleção do próximo PC



Fonte: Elaborado pelos autores com base em Patterson e Hennessy (2017).

O Quadro 4 resume, para cada tipo de instrução usada no programa, de onde vem o próximo valor do PC e se algum registrador é escrito.

Quadro 4 – Fonte do próximo PC e registrador escrito por classe de instrução

Classe	Exemplo na PoC	Próximo PC	Escreve em rd?
Aritmética / memória	`addi`, `lw`, `sw`, `mul`	PC + 4	`addi`, `lw`, `mul`: sim; `sw`: não
Desvio não tomado	`bge a0, zero, fim_while` com n < 0	PC + 4	não
Desvio tomado	`ble a0, t1, if_caso_base` com n ≤ 1	PC + imm	não
Salto direto	`j laco_while` = `jal x0, -48`	PC + imm	não (rd = x0)
Chamada	`jal ra, fatorial`	PC + imm	ra ← PC + 4
Retorno	`jr ra` = `jalr x0, 0(ra)`	R[ra] + 0	não (rd = x0)

3.3 DAS ESTRUTURAS DE ALTO NÍVEL AO ASSEMBLY

O Quadro 5 mostra como traduzimos cada estrutura. Uma coisa que chamou nossa atenção é que existem duas maneiras de montar um desvio condicional, e o programa usa as duas. No *while*, o desvio é tomado para sair do laço quando a condição de repetição deixa de valer (n ≥ 0). Enquanto ela vale, o programa segue em frente, mostra a mensagem de erro e um salto incondicional no final volta para o começo. No *if/else* do fatorial acontece o contrário: o desvio é tomado para entrar no caso base (n ≤ 1), enquanto o *else*, que é a chamada recursiva, fica no caminho sequencial e termina com `j epilogo` para não cair no caso base.

Quadro 5 – Padrões de tradução de estruturas de controle de C para RISC-V

Construção em C	Assembly RISC-V (PoC)	Mecanismo

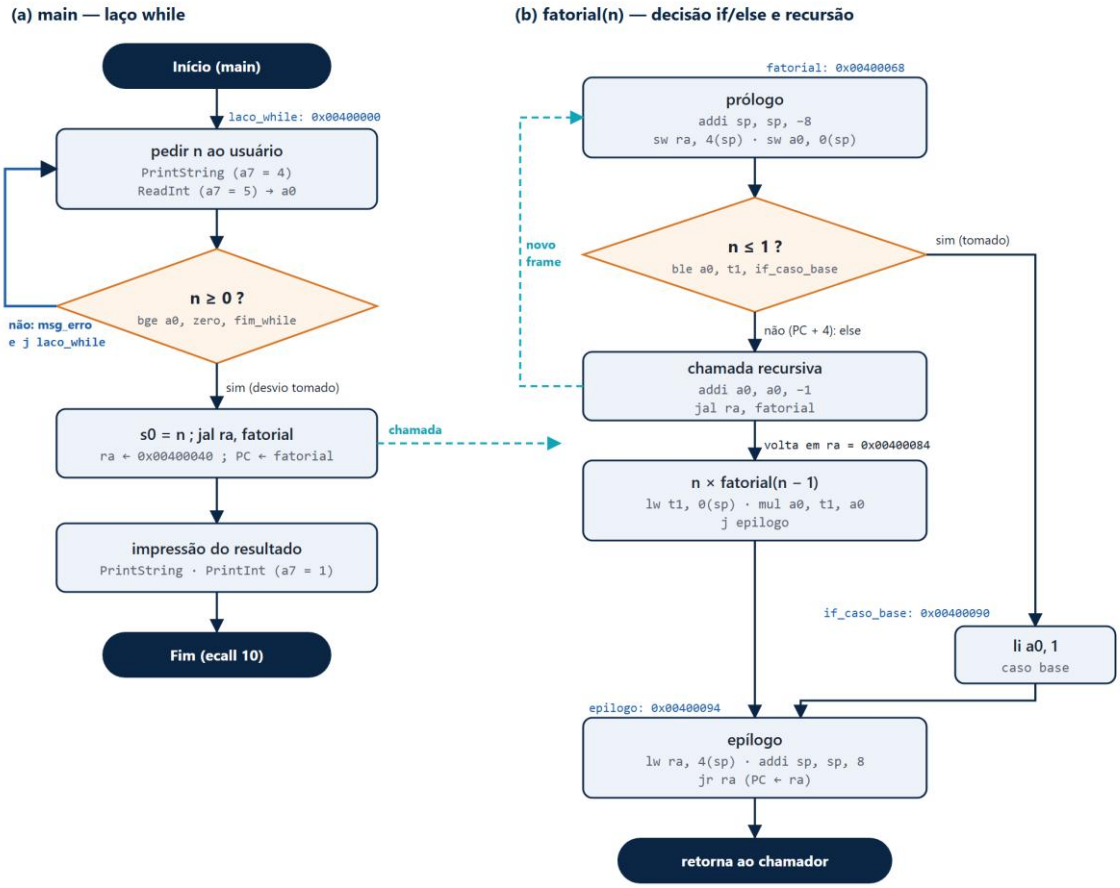
`n = ler();`
`while (n < 0) {`
`  erro(); n = ler();`
`}`
	`laco_while: ...ecall (a7 = 5)`
	`  bge a0, zero, fim_while`
	`  ...ecall (msg_erro)`
	`  j laco_while`
	`fim_while:`
		desvio tomado para sair + salto para trás

`if (n <= 1) r = 1;`
`else r = n * fat(n-1);`
	`ble a0, t1, if_caso_base`
	`  addi · jal · lw · mul`
	`  j epilogo`
	`if_caso_base: li a0, 1`
	`epilogo:`
		desvio tomado para o *then*; *else* no caminho sequencial

`r = fatorial(n);`	`jal ra, fatorial` `mv t0, a0`	argumento em a0, ligação em ra, retorno em a0
`return r;`	`jr ra (jalr x0, 0(ra))`	PC ← ra

A Figura 3 junta as duas rotinas em um fluxograma, com os rótulos e endereços reais. Os losangos são os desvios condicionais (formato B), a seta azul é o salto que repete o laço e a seta tracejada representa a chamada de função.

Figura 3 – Fluxograma do laço while (main) e da decisão if/else com recursão (fatorial)



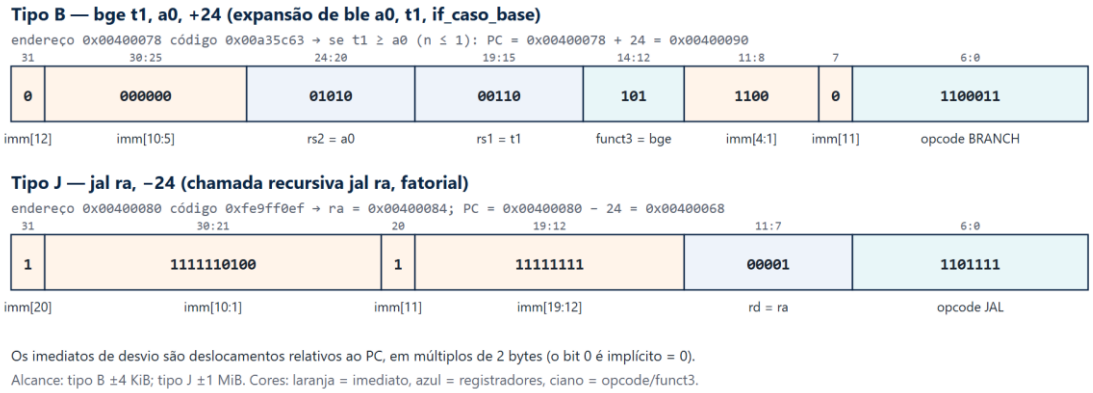
Losango = desvio condicional (tipo B); seta azul = salto incondicional j (jal x0); seta ciano tracejada = chamada (jal ra).
No while, o desvio é tomado para SAIR do laço; no if/else do fatorial, o desvio é tomado para ENTRAR no caso base,
e o bloco else (chamada recursiva) é o caminho sequencial PC + 4.

3.4 CODIFICAÇÃO BINÁRIA DAS INSTRUÇÕES DE DESVIO

Na Figura 4, duas instruções da função `fatorial` aparecem separadas em campos de bits. Como o RISC-V não tem `ble` nativo, o montador transforma `ble a0, t1, if_caso_base` em `bge t1, a0, +24`, apenas trocando a ordem dos operandos (afinal, a0 ≤ t1 é o mesmo que t1 ≥ a0). O imediato de 13 bits fica espalhado em quatro campos, e o bit 0, que é sempre zero, nem chega a ser guardado.

Já a chamada recursiva `jal ra, fatorial`, no endereço `0x00400080`, carrega o deslocamento −24. Fazendo a conta, `0x00400080 − 24` dá `0x00400068`, que é exatamente o rótulo `fatorial`. O campo `rd = 00001` indica que PC + 4, ou seja, `0x00400084`, vai para `ra`. Esse endereço, que é o do `lw t1, 0(sp)`, aparece de novo na pilha (Figura 5) como endereço de retorno das chamadas internas.

Figura 4 – Codificação das instruções bge (formato B) e jal (formato J) da função fatorial



Fonte: Elaborado pelos autores com base em Waterman e Asanović (2019).

3.5 QUADRO DE PILHA DA FUNÇÃO FATORIAL

Cada chamada de `fatorial` reserva 8 bytes na pilha: `ra` fica em `sp+4` e o argumento n em `sp+0`. O n precisa ser guardado porque `a0` é reaproveitado, primeiro para passar n − 1 adiante e depois para receber o resultado. O `ra` também, já que o `jal` interno escreve por cima dele. Optamos por 8 bytes porque é o suficiente para os dois valores e mantém `sp` alinhado em palavra, que é o que o RARS exige. A convenção oficial recomenda 16 bytes, e discutimos essa diferença na seção 5.7.

A Figura 5 mostra a pilha real no momento em que `fatorial(5)` chega ao caso base. São cinco quadros, 40 bytes ao todo, com `sp` em `0x7fffefd4`. O quadro de cima guarda `ra = 0x00400040`, que leva de volta ao `main`, logo depois do `jal ra, fatorial`. Os outros guardam `0x00400084`, a instrução seguinte ao `jal` recursivo. Os valores de n, de 5 até 1, ficam empilhados em ordem e vão sendo recuperados um a um quando a recursão volta.

Figura 5 – Conteúdo da pilha no caso base de fatorial(5)



Instantâneo no caso base de fatorial(5) (rótulo epilogo, a0 = 1). A pilha cresce para endereços menores: cada chamada executa addi sp, sp, −8 e grava ra e n antes de chamar a si mesma (5 frames × 8 B = 40 B). Na volta, lw t1, 0(sp) recarrega n, e o epílogo lê ra (lw ra, 4(sp)) e devolve o espaço (addi sp, sp, 8) antes do jr ra.

4 METODOLOGIA E DESENVOLVIMENTO

4.1 AMBIENTE DE SIMULAÇÃO E MEDIÇÃO

Escrevemos a prova de conceito (PoC) em Assembly RISC-V e a executamos no RARS 1.6, um simulador educacional derivado do MARS (VOLLMAR; SANDERSON, 2006). Com ele é possível montar o código, ver as pseudo-instruções já expandidas, executar passo a passo e acompanhar, a cada instrução, os registradores, o PC e o conteúdo da memória.

O RARS, porém, não conta ciclos de *clock*. Para chegar a esses números, o grupo escreveu um rastreador em Python, o `rv32_tracer.py`. Ele monta o mesmo arquivo `.asm`, expande as pseudo-instruções do mesmo jeito que o RARS (por isso os endereços e as contagens batem) e anota, para cada instrução executada, o PC, a instrução, os registradores alterados e os acessos à memória. As leituras de teclado (`ecall` 5) recebem valores fixos, definidos antes da execução.

A partir desse rastro, estimamos os ciclos com dois modelos descritos por Patterson e Hennessy (2017). No modelo multiciclo, `lw` leva 5 ciclos, `sw`, operações da ULA e `mul` levam 4, e desvios e saltos levam 3. No modelo de *pipeline* de cinco estágios com adiantamento (*forwarding*), cada dependência *load-use* custa 1 ciclo de bolha e cada desvio tomado ou salto custa 2 ciclos descartados, supondo que o processador sempre aposte em "não tomado" e só descubra o resultado no estágio EX. O Quadro 6 resume o ambiente usado.

Quadro 6 – Ambiente de desenvolvimento, simulação e medição

Item	Especificação
Arquitetura (ISA)	RISC-V RV32I + extensão M (apenas para `mul`), 32 bits
Simulador	RARS 1.6, configuração de memória padrão, pseudo-instruções habilitadas
Ambiente de execução	Java SE 8; sistema operacional Windows 11
Rastreador e métricas	`poc/tracer/rv32_tracer.py` (Python 3), desenvolvido pelo grupo
Programas da PoC	`poc/fatorial_completo_tema4.s` (principal), `poc/fatorial_iterativo.asm` (versão iterativa, para comparação) e `poc/erros_comuns.asm` (falhas)
Saídas geradas	`poc/resultados/resultados.json` e `trace_programa.csv` (rastro de todas as instruções)
Validação cruzada	resultados e contagem de instruções conferidos no próprio RARS 1.6 (opção `ic` da linha de comando)

4.2 ESTRUTURA DA PROVA DE CONCEITO

O programa principal, `fatorial_completo_tema4.s` (Apêndice A), tem três partes, uma para cada estrutura pedida no tema:

a) **Parte 1, laço while:** pede um número n pelo console (`ecall` com `a7` = 5) e, enquanto ele for negativo, mostra uma mensagem de erro e pede de novo;
b) **Parte 2, if/else:** dentro da função `fatorial`, testa o caso base (n ≤ 1) para decidir entre devolver 1 e continuar a recursão;
c) **Parte 3, recursão e pilha:** `fatorial(n)` chama `fatorial(n – 1)`, e cada chamada abre um quadro de 8 bytes na pilha.

No final, o resultado é impresso no console. Na execução que usamos como referência, digitamos -3, -1, 5: as duas primeiras entradas são recusadas pelo laço, cada uma com a mensagem de erro, e a terceira é aceita, e o programa imprime 120. Digitando 13, aparece o estouro de 32 bits comentado na seção 4.5. Para comparar a recursão com um laço, escrevemos um segundo programa, `fatorial_iterativo.asm` (Apêndice B), que calcula o mesmo fatorial com uma função folha, sem usar a pilha. Há ainda um terceiro, `erros_comuns.asm` (Apêndice C), feito só para reproduzir os erros típicos do tema.

4.3 PARTE 1: LAÇO WHILE

O laço mostra a mensagem, lê o valor digitado (que o sistema coloca em `a0`) e testa o sinal com `bge a0, zero, fim_while`. A ULA compara `a0` com `x0`: se `a0` for maior ou igual a zero, o PC recebe o endereço de `fim_while`; se não, segue para PC + 4, que é o corpo do laço. Ali o programa guarda o valor lido em `s1`, mostra a mensagem de erro e o `j laco_while` manda tudo de volta para o começo. Em C, o código equivale a `n = ler(); while (n < 0) { erro(); n = ler(); }`.

```
laco_while:
    li a7, 4                # Imprime mensagem de input
    la a0, msg_input
    ecall

    li a7, 5                # Lê inteiro do teclado (fica no a0)
    ecall

    # Condição do While
    bge a0, zero, fim_while # Se a0 >= 0 (Branch if Greater or Equal), quebra o laço e vai para
'fim_while'

    # Corpo do While (se a0 < 0)
    mv s1, a0               # Guarda o valor temporariamente
    li a7, 4                # Imprime mensagem de erro
    la a0, msg_erro
    ecall
    j laco_while            # Salto incondicional voltando para o topo do laço (Loop)

fim_while:
```

A Tabela 1 traz o rastro das três passagens pelo laço. Nas duas primeiras o número lido é negativo (o −3 aparece como `0xfffffffd`, em complemento de dois), então o `bge` não é tomado, a mensagem de erro aparece e o `j` em `0x00400030` leva o PC de volta a `0x00400000`. Na terceira, com o valor 5, o desvio é tomado e o PC pula de `0x00400018` para `0x00400034`.

Tabela 1 – Rastro das três passagens pelo laço while (entradas −3, −1 e 5)

Passo	PC	Instrução básica	Efeito
1	`0x00400000`	`addi a7, zero, 4`	`a7=0x00000004`
2	`0x00400004`	`auipc a0, 0xfc10`	`a0=0x10010004`
3	`0x00400008`	`addi a0, a0, -4`	`a0=0x10010000`
4	`0x0040000c`	`ecall`	PrintString: exibe msg_input
5	`0x00400010`	`addi a7, zero, 5`	`a7=0x00000005`
6	`0x00400014`	`ecall`	ReadInt: a0 ← -3 (digitado)
7	`0x00400018`	`bge a0, zero, +28`	n = -3 < 0: não tomado (PC + 4)
8	`0x0040001c`	`add s1, zero, a0`	`s1=0xfffffffd`
9	`0x00400020`	`addi a7, zero, 4`	`a7=0x00000004`
10	`0x00400024`	`auipc a0, 0xfc10`	`a0=0x10010024`
11	`0x00400028`	`addi a0, a0, 9`	`a0=0x1001002d`
12	`0x0040002c`	`ecall`	PrintString: exibe msg_erro
13	`0x00400030`	`jal zero, -48`	PC ← laco_while (0x00400000)
14	`0x00400000`	`addi a7, zero, 4`	—
15	`0x00400004`	`auipc a0, 0xfc10`	`a0=0x10010004`
16	`0x00400008`	`addi a0, a0, -4`	`a0=0x10010000`
17	`0x0040000c`	`ecall`	PrintString: exibe msg_input
18	`0x00400010`	`addi a7, zero, 5`	`a7=0x00000005`
19	`0x00400014`	`ecall`	ReadInt: a0 ← -1 (digitado)
20	`0x00400018`	`bge a0, zero, +28`	n = -1 < 0: não tomado (PC + 4)
21	`0x0040001c`	`add s1, zero, a0`	`s1=0xffffffff`

Passo	PC	Instrução básica	Efeito
22	0x00400020	addi a7, zero, 4	a7=0x00000004
23	0x00400024	auipc a0, 0xfc10	a0=0x10010024
24	0x00400028	addi a0, a0, 9	a0=0x1001002d
25	0x0040002c	ecall	PrintString: exibe msg_erro
26	0x00400030	jal zero, -48	PC ← laco_while (0x00400000)
27	0x00400000	addi a7, zero, 4	—
28	0x00400004	auipc a0, 0xfc10	a0=0x10010004
29	0x00400008	addi a0, a0, -4	a0=0x10010000
30	0x0040000c	ecall	PrintString: exibe msg_input
31	0x00400010	addi a7, zero, 5	a7=0x00000005
32	0x00400014	ecall	ReadInt: a0 ← 5 (digitado)
33	0x00400018	bge a0, zero, +28	n = 5 ≥ 0: tomado → fim_while

4.4 PARTES 2 E 3: IF/ELSE, RECURSÃO E A VERSÃO ITERATIVA

A função recursiva tem quatro partes: prólogo, teste do caso base, chamada recursiva e epílogo. O *if/else* é a instrução `ble a0, t1, if_caso_base`, com `t1` valendo 1. Se n ≤ 1, o desvio é tomado, e o caso base coloca 1 em `a0` e cai direto no epílogo. Caso contrário, o programa segue para o *else* e chama `fatorial(n - 1)`. Quando essa chamada volta, o valor de n já não está em nenhum registrador, por isso ele é lido de novo da pilha com `lw t1, 0(sp)` antes da multiplicação. O `j epilogo` no fim do *else* serve para não executar o caso base por engano.

```
fatorial:
    # Prólogo: Empilhamento (Stack Frame)
    addi sp, sp, -8         # Aloca 8 bytes na pilha (Stack Pointer)
    sw ra, 4(sp)            # Salva o endereço de retorno na pilha
    sw a0, 0(sp)            # Salva o valor atual de 'n' na pilha

    # ========================================================================
    # 3. ESTRUTURA DE DECISÃO (Equivalente a IF/ELSE)
    # Lógica: if (n <= 1) { return 1; } else { return n * fatorial(n-1); }
    # ========================================================================
    li t1, 1
    ble a0, t1, if_caso_base # IF: Se a0 <= 1, desvia para 'if_caso_base'

    # ELSE (Corpo da recursão)
    addi a0, a0, -1         # Decrementa n
    jal ra, fatorial        # Chamada recursiva

    # Retorno do Else (Desempilhamento parcial)
    lw t1, 0(sp)            # Recupera o 'n' empilhado
    mul a0, t1, a0          # n * fatorial(n-1)
    j epilogo               # Pula o if_caso_base

if_caso_base:
    li a0, 1                # Retorna 1

epilogo:
```

```
    # Epílogo: Desempilhamento de dados (Stack Frame)
    lw ra, 4(sp)            # Recupera o endereço de retorno
    addi sp, sp, 8          # Libera os 8 bytes da pilha
    jr ra                   # Retorna ao chamador
```

A versão iterativa, que está no arquivo `fatorial_iterativo.asm`, faz a mesma conta usando um acumulador em `t0`. Como não chama nenhuma outra função, não precisa salvar `ra` nem mexer em `sp`:

```
fatorial_it:
    li t0, 1                # t0 = acumulador = 1
    li t1, 1                # t1 = limite
fit_laco:
    ble a0, t1, fit_fim     # while (n > 1): se n <= 1, sai do laço
    mul t0, t0, a0          # acc *= n
    addi a0, a0, -1         # n--
    j fit_laco              # volta para o teste
fit_fim:
    mv a0, t0               # valor de retorno em a0
    jr ra                   # retorna ao chamador
```

Para acompanhar a pilha passo a passo, executamos a função recursiva com n = 3 no mesmo contexto do `main` (`ra = 0x00400040`). Foram 35 instruções e 11 acessos à pilha no total (o rastro completo está no Apêndice D), e a Tabela 2 mostra só os momentos de chamada e retorno. Dá para ver `sp` descendo 8 bytes a cada chamada e subindo 8 a cada retorno até voltar a `0x7fffeffc`, o que garante que o `main` encontre a pilha do jeito que deixou. Também aparece `t1` recebendo da pilha o n de cada nível logo antes de cada `mul`.

Tabela 2 – Estado dos registradores nos eventos de chamada e retorno de fatorial(3)

Passo	Evento	sp	ra	a0	t1
3	prólogo de fatorial(3) concluído	0x7fffeff4	0x00400040	3	0
7	jal: chamada a fatorial(2)	0x7fffeff4	0x00400084	2	1
10	prólogo de fatorial(2) concluído	0x7fffefec	0x00400084	2	1
14	jal: chamada a fatorial(1)	0x7fffefec	0x00400084	1	1
17	prólogo de fatorial(1) concluído	0x7fffefe4	0x00400084	1	1
20	caso base: a0 ← 1	0x7fffefe4	0x00400084	1	1
23	jr ra → 0x00400084 (volta a fatorial(2))	0x7fffefec	0x00400084	1	1
25	mul: a0 = 2 × 1	0x7fffefec	0x00400084	2	2
29	jr ra → 0x00400084 (volta a fatorial(3))	0x7fffeff4	0x00400084	2	2
31	mul: a0 = 3 × 2	0x7fffeff4	0x00400084	6	3
35	jr ra → 0x00400040 (volta ao main)	0x7fffeffc	0x00400040	6	3

4.5 ESTOURO SILENCIOSO DE 32 BITS

A instrução `mul` guarda em `rd` só os 32 bits menos significativos do produto. Quando digitamos 13, o resultado correto seria 6.227.020.800, mas o maior inteiro

positivo que cabe em 32 bits é 2.147.483.647. As duas versões, a recursiva e a iterativa, imprimiram 1.932.053.504, que é exatamente 6.227.020.800 − 2^{32}, o resto da divisão por 2^{32}. Não houve exceção nem aviso de nenhum tipo, e o programa simplesmente continuou com o valor errado.

4.6 REPRODUÇÃO DOS DESAFIOS COMUNS

O programa `erros_comuns.asm` escolhe uma de três falhas conforme o valor carregado em `s11`. Rodamos cada uma no rastreador com um limite de 200.000 instruções, para conseguir identificar os laços que não terminam. O Quadro 7 resume o que aconteceu em cada caso.

Quadro 7 – Falhas reproduzidas, sintomas observados e correções

Cenário	Código com erro	Sintoma observado	Correção
1. Condição de saída invertida	`bge a0, zero, fim1` com a0 = −1 e `addi a0, a0, -1` no corpo	PC preso no laço em `0x00400024`; a0 = −66.667 após 200.000 instruções; só terminaria após ~2^{31} iterações, quando o valor "dá a volta"	`ble a0, zero, fim1` (sair quando a0 ≤ 0)
2. `ra` não preservado	função recursiva `conta` sem salvar `ra` antes do `jal` interno	`ra` = `0x00400044` aponta para o próprio `ret`; PC preso em `0x00400044` indefinidamente	salvar `ra` no prólogo e restaurá-lo no epílogo
3. Pilha desalinhada	`addi sp, sp, -6` seguido de `sw ra, 0(sp)`	exceção de alinhamento no 8º passo: endereço `0x7fffeff6` não é múltiplo de 4	reservar múltiplos de 4 bytes (8, como na PoC, ou 16, como na psABI)

4.7 PROCEDIMENTO DE EXECUÇÃO NO RARS

No RARS, o programa é aberto em File › Open, montado com Run › Assemble (F3) e executado de uma vez com Run › Go (F5) ou passo a passo com Run › Step (F7). Os valores de n são digitados na janela *Run I/O*. A aba *Text Segment* mostra, para cada instrução, o endereço, o código de máquina, a instrução básica e a linha original, que são os mesmos dados da Figura 4. Para ver a pilha, basta escolher "current sp" na janela *Data Segment*. As métricas podem ser conferidas em Tools › Instruction Counter (total de instruções) e Tools › Instruction Statistics (instruções por categoria). O roteiro completo da demonstração está no Apêndice E.

Para conferir o rastreador, rodamos os mesmos programas no RARS 1.6 pela linha de comando, que mostra a saída do console, o total de instruções executadas (opção `ic`) e o valor final dos registradores pedidos. A Figura 6 mostra a execução

completa com as entradas −3, −1 e 5. Na Figura 7, a execução foi interrompida no 44º passo, que é o momento em que `fatorial(5)` chega ao caso base, e o trecho da pilha foi salvo com a opção `dump`. Os valores lidos da memória do RARS são os mesmos da Figura 5.

Figura 6 – Execução completa de fatorial_completo_tema4.s no RARS 1.6 com as entradas −3, −1 e 5: console, total de instruções e registradores

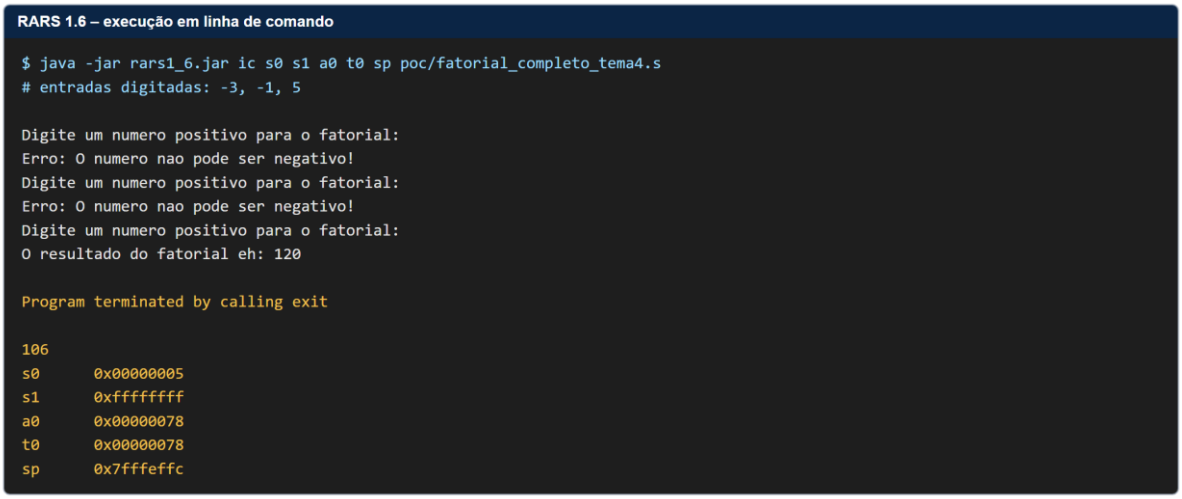
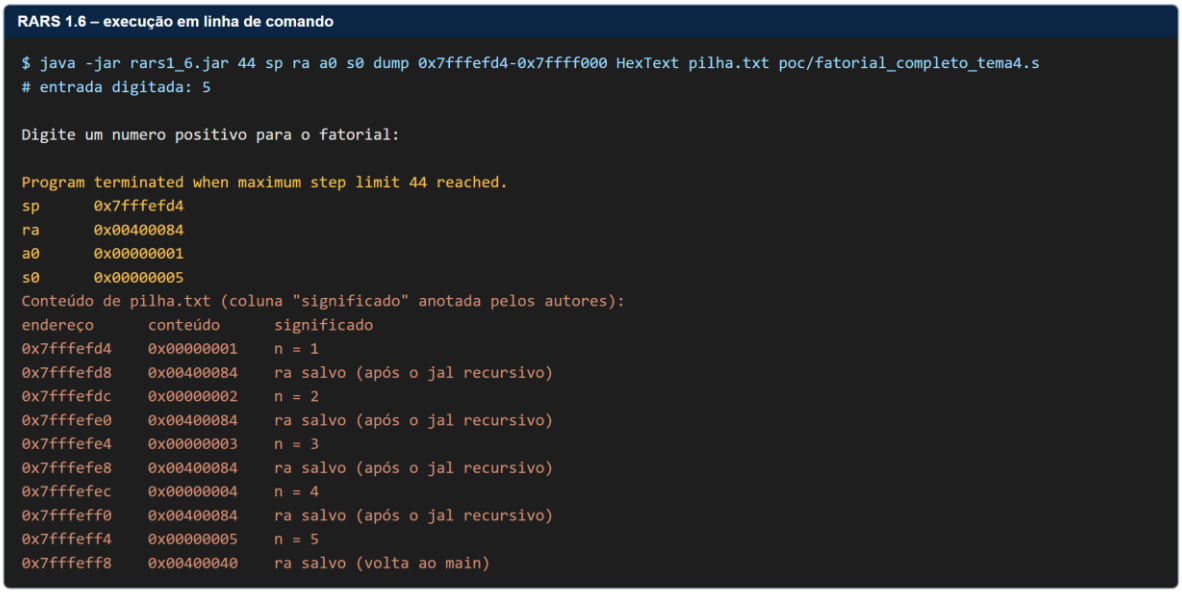


Figura 7 – Pilha no caso base de fatorial(5) lida da memória do RARS 1.6 (execução interrompida no 44º passo)



5 DISCUSSÃO DOS RESULTADOS

5.1 VALIDAÇÃO FUNCIONAL

A Tabela 3 compara o que esperávamos com o que foi obtido. Todos os valores bateram, inclusive o 13! truncado, que confirma o comportamento descrito na seção 4.5.

Tabela 3 – Resultados esperados e obtidos na execução dos programas

Verificação	Onde conferir	Esperado	Obtido
Entradas −3 e −1 rejeitadas	`bge` do *while* / `msg_erro`	2 mensagens de erro	2 mensagens de erro
Entrada 5 aceita	`s0`	5	5
fatorial recursivo(5)	console / `a0`	120	120
fatorial iterativo(5)	console de `fatorial_iterativo.asm`	120	120
fatorial(13) em 32 bits	console dos dois programas	$6.227.020.800 \bmod 2^{32} = 1.932.053.504$	1.932.053.504 / 1.932.053.504

5.2 PERFIL DE INSTRUÇÕES DO PROGRAMA

Com as entradas −3, −1 e 5, o programa inteiro executou 107 instruções, distribuídas por classe na Tabela 4. O RARS mostrou 106 (Figura 6), uma a menos, porque o contador dele não inclui o `ecall` que encerra o programa: num teste com três instruções terminando em `ecall` 10, ele também contou só duas. Tirando essa diferença fixa, as contagens bateram em todas as execuções que conferimos. Desvios condicionais (8) e saltos (16) somam 22,4% do total. Dos 8 desvios, 2 foram tomados e 6 não. Juntando os saltos, o PC mudou de caminho 18 vezes, mais ou menos uma vez a cada 6 instruções.

Tabela 4 – Instruções executadas por classe e ciclos no modelo multiciclo

Classe	Quantidade	%	Ciclos/instr.	Ciclos
ULA (`add`, `addi`)	43	40,2%	4	172
Multiplicação (`mul`)	4	3,7%	4	16
Carga (`lw`)	9	8,4%	5	45
Armazenamento (`sw`)	10	9,3%	4	40
Desvio condicional (tipo B)	8	7,5%	3	24
Salto (`jal`, `jalr`)	16	15%	3	48
`auipc` / `lui`	6	5,6%	4	24
Chamada de sistema (`ecall`)	11	10,3%	4	44
Total	**107**	**100%**	**CPI 3,86**	**413**

Fonte: Elaborado pelos autores com base no modelo multiciclo de Patterson e Hennessy (2017).

No modelo multiciclo, foram 413 ciclos (CPI 3,86). No *pipeline* de cinco estágios, 151 ciclos (CPI 1,411), que se dividem assim: 107 ciclos de vazão ideal, 4 para encher o *pipeline*, 4 bolhas *load-use* e 36 ciclos perdidos com os redirecionamentos do PC. Em outras palavras, cerca de 23,8% dos ciclos foram gastos só por causa do fluxo de controle. Esse número ajuda a entender por que os processadores reais investem tanto em previsão de desvios.

5.3 RECURSÃO VERSUS ITERAÇÃO

A Tabela 5 e a Figura 8 comparam as duas versões do fatorial para vários valores de n. As medições formaram retas perfeitas, e cada fórmula pode ser explicada contando as instruções do código:

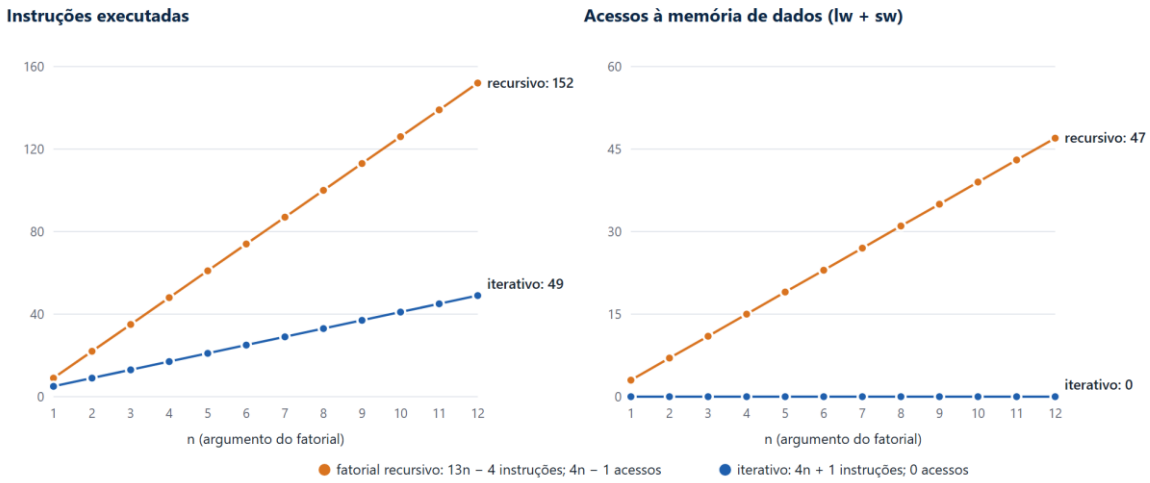
- na versão recursiva, cada nível que não é o caso base executa 13 instruções (3 no prólogo, `li` e `ble` no teste, `addi` e `jal` na chamada, `lw`, `mul` e `j` depois do retorno e 3 no epílogo), e o caso base executa 9. Isso dá 13(n − 1) + 9 = **13n − 4 instruções**;
- cada nível faz 2 `sw` no prólogo e 1 `lw` no epílogo, e os níveis que não são o caso base fazem mais um `lw` para recuperar n. São 4 acessos por nível e 3 no caso base, ou seja, **4n − 1 acessos à memória**;
- a pilha cresce **8 bytes por nível**, um quadro para cada chamada ativa;
- a versão iterativa executa 2 instruções de preparação, 4 por volta do laço (são n − 1 voltas) e 3 na saída, o que dá **4n + 1 instruções**, sem nenhum acesso à memória de dados.

Tabela 5 – Custo do fatorial recursivo e iterativo em função de n

n	Versão	Instruções	lw + sw	Acessos totais	Pilha	Ciclos multiciclo	Ciclos pipeline
1	recursivo	9	3	12	8 B	35	17
	iterativo	5	0	5	0 B	18	13
3	recursivo	35	11	46	24 B	135	57
	iterativo	13	0	13	0 B	46	25
5	recursivo	61	19	80	40 B	235	97
	iterativo	21	0	21	0 B	74	37
10	recursivo	126	39	165	80 B	485	197
	iterativo	41	0	41	0 B	144	67
12	recursivo	152	47	199	96 B	585	237
	iterativo	49	0	49	0 B	172	79

Nota: acessos totais = busca de instruções + acessos à memória de dados.

Figura 8 – Instruções executadas e acessos à memória de dados em função de n



Para n = 3, a recursão executou 35 instruções e fez 11 acessos à pilha, com cerca de 57 ciclos no *pipeline* estimado (CPI 1,629). São os mesmos números que o grupo tinha calculado à mão no roteiro da apresentação, o que serviu como uma boa conferência das duas partes do trabalho.

Para n = 5, a versão recursiva executa 61 instruções contra 21 da iterativa, quase 2,9 vezes mais. Ela também faz 19 acessos à memória de dados (a iterativa não faz nenhum) e, somando a busca das próprias instruções, chega a 80 acessos à memória contra 21. No *pipeline*, são 97 ciclos contra 37. Conforme n cresce, a recursão tende a executar 13/4 = 3,25 vezes mais instruções.

O que achamos mais interessante é que o algoritmo é o mesmo nas duas versões: são sempre n − 1 multiplicações. Toda a diferença vem do controle, isto é,

de salvar e restaurar o contexto, guardar o endereço de retorno e desviar três vezes por nível (`jal`, `j epilogo` e `jr ra`). É justamente esse custo que os compiladores tentam eliminar quando transformam recursões em laços ou expandem funções em linha (AHO et al., 2008).

5.4 DESVIOS, PIPELINE E PREVISÃO DE DESVIOS

Um previsor de desvios só consegue esconder a penalidade se o desvio tiver um comportamento regular. Com n = 5, o `ble` do caso base é executado cinco vezes: não é tomado nas quatro primeiras e é tomado na última. O `ble` do laço iterativo segue o mesmo padrão. Smith (1981) mostrou que padrões assim são previstos com facilidade por contadores saturados de 2 bits.

O desvio do *while* é diferente, porque depende do número que o usuário digita, e o histórico não ajuda a adivinhá-lo. O mesmo acontece com um *if* que depende do conteúdo de um vetor. Se os dados estiverem ordenados, porém, o desvio muda de comportamento uma única vez e fica fácil de prever, e é por isso que percorrer um vetor ordenado pode ser mais rápido, como as próprias orientações da atividade lembram.

O `jr ra` merece um comentário à parte. Ele é um salto indireto, cujo destino vem de um registrador, e na recursão o mesmo `jr ra` volta às vezes para `0x00400084` e às vezes para `0x00400040`. Processadores reais usam uma pequena pilha de endereços de retorno, em hardware, para prever esse tipo de salto (HENNESSY; PATTERSON, 2019). Já as 4 bolhas *load-use* vêm todas do mesmo lugar: o `lw t1, 0(sp)` seguido logo do `mul a0, t1, a0`, uma vez por nível de retorno. Em um processador real, daria para escondê-las colocando alguma instrução independente entre o `lw` e o `mul`.

5.5 ESTOURO ARITMÉTICO E AUSÊNCIA DE FLAGS

No x86, uma multiplicação que estoura liga as *flags* CF e OF, e o programa pode verificar isso com um salto condicional (`jo`). No RISC-V não existe onde registrar o estouro, e por isso o 13! foi truncado sem nenhum aviso. Se for preciso detectar o problema, isso tem que ser feito no próprio código. Para multiplicação com sinal, uma forma é comparar a parte alta do produto (`mulh`) com a extensão de sinal da parte baixa:

```
        mul   t0, a0, a1      # parte baixa do produto (32 bits)
        mulh  t1, a0, a1      # parte alta do produto (bits 63..32)
        srai  t2, t0, 31      # extensão de sinal da parte baixa
        bne   t1, t2, estouro # se diferentes, o produto não cabe em 32 bits
```

São três instruções a mais, mas só nos lugares em que a verificação realmente importa. Esse foi o caminho escolhido pelo RISC-V: um hardware mais simples, sem estado escondido, em troca de verificações explícitas no software (WATERMAN; ASANOVIĆ, 2019).

5.6 FALHAS DE CONTROLE E SEGURANÇA

As três falhas do Quadro 7 têm algo em comum: o processador faz exatamente o que o código manda, sem saber o que o programador queria. Uma condição invertida não gera erro nenhum, só um laço que não acaba. Um `ra` que não foi salvo faz o programa voltar para um endereço válido, só que errado. A única falha que gera exceção na hora é o desalinhamento, porque ela quebra uma regra do próprio hardware de acesso à memória.

O segundo caso também tem um lado de segurança. Se o endereço de retorno guardado na pilha puder ser sobrescrito por dados vindos de fora, por exemplo um vetor local escrito além do seu tamanho, o `jr ra` vai saltar para onde o atacante quiser. Essa é a ideia por trás dos ataques clássicos de estouro de *buffer* na pilha (ALEPH ONE, 1996).

5.7 ALINHAMENTO DO QUADRO DE PILHA E DOS DADOS

Na PoC usamos quadros de 8 bytes, o mínimo para guardar `ra` e n mantendo `sp` múltiplo de 4, que é a única exigência que o RARS confere nos `lw` e `sw`. A convenção oficial, no entanto, pede `sp` alinhado em 16 bytes (WATERMAN; ASANOVIĆ, 2019), para que o código funcione com dados de 64 e 128 bits e com bibliotecas compiladas que contam com esse alinhamento. Em um programa isolado no simulador, como o nosso, 8 bytes funcionam bem e gastam metade da pilha. Se a função fosse misturada com código gerado por um compilador C, porém, seria preciso reservar 16 bytes por quadro, mesmo deixando metade sem uso.

No segmento de dados, o alinhamento não chega a ser um problema no nosso programa, porque ele só guarda mensagens de texto, que são lidas byte a byte: `msg_erro`, por exemplo, começa em `0x1001002d`, um endereço ímpar, e funciona

normalmente. Se o programa precisasse gravar uma palavra (`sw`) logo depois dessas mensagens, porém, seria preciso usar a diretiva `.align 2` antes, senão o endereço poderia não ser múltiplo de 4 e o RARS pararia com erro de endereço desalinhado, que é o "erro de alinhamento de memória" citado nas orientações da atividade.

5.8 LIMITAÇÕES

Alguns cuidados são necessários na leitura desses resultados. O RARS é um simulador funcional, então a contagem de instruções e de acessos à memória é exata, mas os ciclos são estimativas tiradas de modelos de livro-texto, sem cache, sem previsor de desvios e sem execução superescalar. Em um processador real, os $4n - 1$ acessos à pilha da recursão caem em endereços vizinhos e próximos no tempo (localidade espacial e temporal) e provavelmente seriam atendidos pela cache L1. O previsor de desvios também reduziria as penalidades de controle.

Além disso, nosso rastreador só implementa as instruções do RV32IM que usamos no programa. Por fim, o fatorial é um exemplo pequeno e bem comportado. Em algoritmos com mais de uma chamada recursiva por nível, como o Fibonacci ingênuo, o número de chamadas cresce de forma exponencial, e as diferenças seriam bem maiores.

5.9 IMPACTOS NO DESENVOLVIMENTO DE SOFTWARE DE ALTO DESEMPENHO

Para quem desenvolve software, esses resultados viram algumas recomendações práticas. Em trechos críticos de desempenho, vale preferir laços a recursões profundas, ou pelo menos escrever a recursão de cauda para que o compilador consiga convertê-la. Funções pequenas e folha saem mais baratas, porque não usam a pilha. Quando a profundidade da recursão depende de dados externos, é prudente limitá-la para evitar o estouro de pilha. Organizar os dados de forma que os desvios fiquem previsíveis também ajuda. E, em código de baixo nível, seguir à risca a convenção de chamada e as regras de alinhamento evita falhas difíceis de encontrar, como as que reproduzimos no Quadro 7.

6 CONCLUSÃO

Ao longo do trabalho ficou claro que *if*, *while* e chamadas de função não existem no hardware. Tudo isso é construído com poucas instruções, cuja função é escolher o próximo valor do PC. No RISC-V, seis desvios condicionais e dois saltos são suficientes. E, como não há registrador de *flags*, comparar e desviar viram uma coisa só, o que simplifica o *pipeline* mas deixa para o programa a tarefa de detectar estouros, como vimos no cálculo de 13!.

A prova de conceito nos permitiu ver de perto o que o compilador normalmente esconde: o desvio que encerra um laço e o salto que o repete, as duas formas de montar um *if/else*, o endereço de retorno guardado em `ra` pelo `jal` e o quadro de pilha que cada chamada recursiva cria. Tudo isso com endereços e valores reais, que podem ser conferidos no simulador. As medições mostraram o preço dessa abstração: a versão recursiva do fatorial executa 13n − 4 instruções, faz 4n − 1 acessos à memória e usa 8n bytes de pilha, enquanto a iterativa executa 4n + 1 instruções e não acessa a memória de dados.

Algumas lições ficaram para o grupo. A primeira é que todo erro de fluxo de controle, no fim das contas, é um valor errado no PC, seja por uma condição invertida, seja por um endereço de retorno perdido. A segunda é que a convenção de chamada e o alinhamento funcionam como contratos, e quebrá-los quase nunca dá erro na hora, o que torna esses problemas difíceis de achar. A terceira é que o custo do controle pode ser medido e, em trechos curtos de código, chega a pesar mais do que a própria conta. Para quem vai seguir na área de computação, entender isso ajuda a ler o código que o compilador gera, a depurar falhas de baixo nível e a perceber riscos de segurança ligados à pilha.

Como continuação, seria interessante rodar a mesma PoC em um simulador de *pipeline* que conte ciclos de verdade, comparar com o código gerado por um compilador C em diferentes níveis de otimização e testar recursões com mais de uma chamada por nível, como o Fibonacci.

REFERÊNCIAS

AHO, A. V.; LAM, M. S.; SETHI, R.; ULLMAN, J. D. **Compiladores**: princípios, técnicas e ferramentas. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

ALEPH ONE. Smashing the stack for fun and profit. **Phrack Magazine**, v. 7, n. 49, 1996.

ASANOVIĆ, K.; PATTERSON, D. A. **Instruction sets should be free**: the case for RISC-V. Berkeley: EECS Department, University of California, 2014. (Technical Report UCB/EECS-2014-146).

HENNESSY, J. L.; PATTERSON, D. A. **Computer architecture**: a quantitative approach. 6. ed. Cambridge: Morgan Kaufmann, 2019.

PATTERSON, D. A.; HENNESSY, J. L. **Computer organization and design RISC-V edition**: the hardware/software interface. Cambridge: Morgan Kaufmann, 2017.

PATTERSON, D.; WATERMAN, A. **The RISC-V reader**: an open architecture atlas. Berkeley: Strawberry Canyon, 2017.

SMITH, J. E. A study of branch prediction strategies. In: INTERNATIONAL SYMPOSIUM ON COMPUTER ARCHITECTURE, 8., 1981, Minneapolis. **Proceedings** [...]. Los Alamitos: IEEE Computer Society Press, 1981. p. 135-148.

STALLINGS, W. **Arquitetura e organização de computadores**. 10. ed. São Paulo: Pearson Education do Brasil, 2017.

TANENBAUM, A. S.; AUSTIN, T. **Organização estruturada de computadores**. 6. ed. São Paulo: Pearson Prentice Hall, 2013.

VOLLMAR, K.; SANDERSON, P. MARS: an education-oriented MIPS assembly language simulator. **ACM SIGCSE Bulletin**, v. 38, n. 1, p. 239-243, 2006.

WATERMAN, A. **Design of the RISC-V instruction set architecture**. 2016. Tese (Doutorado em Ciência da Computação) – University of California, Berkeley, 2016.

WATERMAN, A.; ASANOVIĆ, K. (ed.). **The RISC-V instruction set manual**: volume I: unprivileged ISA. Document version 20191213. [*S. l.*]: RISC-V Foundation, 2019.

APÊNDICE A – CÓDIGO-FONTE DE FATORIAL_COMPLETO_TEMA4.S

```
# Arquivo: fatorial_completo_tema4.s
# Simulador: RARS (RISC-V)

.data
    msg_input:  .asciz "\nDigite um numero positivo para o fatorial: "
    msg_erro:   .asciz "\nErro: O numero nao pode ser negativo!"
    msg_output: .asciz "\nO resultado do fatorial eh: "

.text
.globl main

main:
    # ================================================================
    # 1. ESTRUTURA DE REPETIÇÃO (Equivalente a WHILE)
    # Lógica: while (input < 0) { print erro; pede de novo; }
    # ================================================================
laco_while:
    li a7, 4                # Imprime mensagem de input
    la a0, msg_input
    ecall

    li a7, 5                # Lê inteiro do teclado (fica no a0)
    ecall

    # Condição do While
    bge a0, zero, fim_while # Se a0 >= 0 (Branch if Greater or Equal), quebra o laço e vai para
'fim_while'

    # Corpo do While (se a0 < 0)
    mv s1, a0               # Guarda o valor temporariamente
    li a7, 4                # Imprime mensagem de erro
    la a0, msg_erro
    ecall
    j laco_while            # Salto incondicional voltando para o topo do laço (Loop)

fim_while:
    mv s0, a0               # Salva o input validado no registrador s0

    # Configuração e Chamada do Procedimento
    mv a0, s0               # Passa 'n' como argumento
    jal ra, fatorial        # Salta para a recursão, salvando o endereço de retorno em 'ra'

    # Saída de Dados
    mv t0, a0               # Guarda o resultado

    li a7, 4
    la a0, msg_output
    ecall

    mv a0, t0
    li a7, 1                # Imprime o resultado inteiro
    ecall

    li a7, 10               # Encerra o programa
    ecall

# ================================================================
# 2. CHAMADA RECURSIVA SIMPLES E MANIPULAÇÃO DA PILHA (Stack Frame)
# ================================================================
fatorial:
    # Prólogo: Empilhamento (Stack Frame)
    addi sp, sp, -8         # Aloca 8 bytes na pilha (Stack Pointer)
    sw ra, 4(sp)            # Salva o endereço de retorno na pilha
    sw a0, 0(sp)            # Salva o valor atual de 'n' na pilha
```

```
    # ======================================================================
    # 3. ESTRUTURA DE DECISÃO (Equivalente a IF/ELSE)
    # Lógica: if (n <= 1) { return 1; } else { return n * fatorial(n-1); }
    # ======================================================================
    li t1, 1
    ble a0, t1, if_caso_base # IF: Se a0 <= 1, desvia para 'if_caso_base'

    # ELSE (Corpo da recursão)
    addi a0, a0, -1         # Decrementa n
    jal ra, fatorial        # Chamada recursiva

    # Retorno do Else (Desempilhamento parcial)
    lw t1, 0(sp)            # Recupera o 'n' empilhado
    mul a0, t1, a0          # n * fatorial(n-1)
    j epilogo               # Pula o if_caso_base

if_caso_base:
    li a0, 1                # Retorna 1

epilogo:
    # Epílogo: Desempilhamento de dados (Stack Frame)
    lw ra, 4(sp)            # Recupera o endereço de retorno
    addi sp, sp, 8          # Libera os 8 bytes da pilha
    jr ra                   # Retorna ao chamador
```

APÊNDICE B – CÓDIGO-FONTE DE FATORIAL_ITERATIVO.ASM

```
# Arquivo: fatorial_iterativo.asm
# Simulador: RARS (RISC-V)
#
# Versão ITERATIVA do fatorial, usada só para comparar com a versão
# recursiva de fatorial_completo_tema4.s (seção 5.3 do relatório e
# slide 5 da apresentação). A função fatorial_it é folha: não chama
# ninguém, então não precisa salvar ra nem mexer na pilha.

.data
    msg_input:  .asciz "\nDigite um numero positivo para o fatorial: "
    msg_output: .asciz "\nO resultado do fatorial (iterativo) eh: "

.text
.globl main

main:
    li a7, 4                # Imprime mensagem de input
    la a0, msg_input
    ecall

    li a7, 5                # Lê inteiro do teclado (fica no a0)
    ecall

    jal ra, fatorial_it     # a0 = fatorial_it(n)
    mv t2, a0               # Guarda o resultado

    li a7, 4
    la a0, msg_output
    ecall

    mv a0, t2
    li a7, 1                # Imprime o resultado inteiro
    ecall

    li a7, 10               # Encerra o programa
    ecall

# ========================================================================
# FATORIAL ITERATIVO (laço while, sem pilha)
# Lógica: acc = 1; while (n > 1) { acc *= n; n--; } return acc;
# ========================================================================
fatorial_it:
    li t0, 1                # t0 = acumulador = 1
    li t1, 1                # t1 = limite
fit_laco:
    ble a0, t1, fit_fim     # while (n > 1): se n <= 1, sai do laço
    mul t0, t0, a0          # acc *= n
    addi a0, a0, -1         # n--
    j fit_laco              # volta para o teste
fit_fim:
    mv a0, t0               # valor de retorno em a0
    jr ra                   # retorna ao chamador
```

APÊNDICE C – CÓDIGO-FONTE DE ERROS_COMUNS.ASM

```
# ========================================================================
#  UCB - Arquitetura de Computadores - Tema 4
#  erros_comuns.asm : reprodução controlada dos "desafios comuns"
#
#  Altere o valor carregado em s11 para escolher a falha:
#    1 = laço (quase) infinito por condição de saída invertida
#    2 = sub-rotina que não preserva ra  -> retorno para o lugar errado
#    3 = ponteiro de pilha desalinhado   -> exceção de acesso à memória
#  Simulador: RARS 1.6 (pare a execução manualmente nos casos 1 e 2)
# ========================================================================
        .text
        .globl main
main:
        li    s11, 1              # <<< escolha o cenário (1, 2 ou 3)
        li    t6, 1
        beq   s11, t6, erro1
        li    t6, 2
        beq   s11, t6, erro2
        j     erro3

# ------------------------------------------------------------------------
# ERRO 1 - while (a0 > 0) a0--;  escrito com a condição invertida.
#   Correto  : ble a0, zero, fim   (sai quando a0 <= 0)
#   Errado   : bge a0, zero, fim   (sai quando a0 >= 0)
#   Com a0 = -1 o laço decrementa um número que já é negativo e só
#   termina após ~2^31 iterações, quando o valor "dá a volta" (wrap-around)
#   para positivo. RISC-V não tem flag de overflow que interrompa o laço.
# ------------------------------------------------------------------------
erro1:
        li    a0, -1
laco1:
        bge   a0, zero, fim1      # BUG: condição invertida
        addi  a0, a0, -1
        j     laco1
fim1:
        j     sair

# ------------------------------------------------------------------------
# ERRO 2 - função recursiva que NÃO salva ra na pilha.
#   O "jal ra, conta" interno sobrescreve ra. Quando o caso base executa
#   "ret", ele volta para dentro de conta; o "ret" seguinte usa o MESMO ra
#   e o PC fica preso para sempre em 0x...  (laço infinito de retorno).
# ------------------------------------------------------------------------
erro2:
        li    a0, 3
        jal   ra, conta
        j     sair
conta:
        beq   a0, zero, conta_fim # caso base
        addi  a0, a0, -1
        jal   ra, conta           # BUG: ra do chamador foi perdido
conta_fim:
        ret

# ------------------------------------------------------------------------
# ERRO 3 - frame com tamanho que não é múltiplo de 4.
#   sp passa a 0x7fffeff6; o "sw" seguinte gera a exceção
#   "Store address not aligned on word boundary" no RARS.
# ------------------------------------------------------------------------
erro3:
        addi  sp, sp, -6          # BUG: deveria ser -8 ou -16
        sw    ra, 0(sp)

sair:
        li    a7, 10
        ecall
```

APÊNDICE D – RASTRO COMPLETO DE FATORIAL(3)

Execução de `fatorial(3)` a partir do contexto do `main` (sp = `0x7fffeffc`, ra = `0x00400040`). A coluna "Efeito" lista os registradores alterados e o acesso à memória realizado pela instrução. A Tabela 6 mostra cada uma das instruções executadas, na ordem em que aconteceram.

Tabela 6 – Rastro instrução a instrução de fatorial(3)

Passo	PC	Instrução básica	Efeito
1	0x00400068	addi sp, sp, -8	sp=0x7fffeff4
2	0x0040006c	sw ra, 4(sp)	grava [0x7fffeff8]=0x00400040
3	0x00400070	sw a0, 0(sp)	grava [0x7fffeff4]=0x00000003
4	0x00400074	addi t1, zero, 1	t1=0x00000001
5	0x00400078	bge t1, a0, +24	–
6	0x0040007c	addi a0, a0, -1	a0=0x00000002
7	0x00400080	jal ra, -24	ra=0x00400084
8	0x00400068	addi sp, sp, -8	sp=0x7fffefec
9	0x0040006c	sw ra, 4(sp)	grava [0x7fffeff0]=0x00400084
10	0x00400070	sw a0, 0(sp)	grava [0x7fffefec]=0x00000002
11	0x00400074	addi t1, zero, 1	–
12	0x00400078	bge t1, a0, +24	–
13	0x0040007c	addi a0, a0, -1	a0=0x00000001
14	0x00400080	jal ra, -24	–
15	0x00400068	addi sp, sp, -8	sp=0x7fffefe4
16	0x0040006c	sw ra, 4(sp)	grava [0x7fffefe8]=0x00400084
17	0x00400070	sw a0, 0(sp)	grava [0x7fffefe4]=0x00000001
18	0x00400074	addi t1, zero, 1	–
19	0x00400078	bge t1, a0, +24	–
20	0x00400090	addi a0, zero, 1	–
21	0x00400094	lw ra, 4(sp)	lê [0x7fffefe8]=0x00400084
22	0x00400098	addi sp, sp, 8	sp=0x7fffefec
23	0x0040009c	jalr zero, 0(ra)	–
24	0x00400084	lw t1, 0(sp)	t1=0x00000002; lê [0x7fffefec]=0x00000002
25	0x00400088	mul a0, t1, a0	a0=0x00000002
26	0x0040008c	jal zero, +8	–
27	0x00400094	lw ra, 4(sp)	lê [0x7fffeff0]=0x00400084
28	0x00400098	addi sp, sp, 8	sp=0x7fffeff4
29	0x0040009c	jalr zero, 0(ra)	–
30	0x00400084	lw t1, 0(sp)	t1=0x00000003; lê [0x7fffeff4]=0x00000003
31	0x00400088	mul a0, t1, a0	a0=0x00000006
32	0x0040008c	jal zero, +8	–
33	0x00400094	lw ra, 4(sp)	ra=0x00400040; lê [0x7fffeff8]=0x00400040
34	0x00400098	addi sp, sp, 8	sp=0x7fffeffc
35	0x0040009c	jalr zero, 0(ra)	–

APÊNDICE E – ROTEIRO DA DEMONSTRAÇÃO PRÁTICA (POC)

Roteiro sugerido para a demonstração ao vivo e para a gravação de contingência recomendada nas orientações da atividade. Duração estimada: 6 a 8 minutos.

Preparação

1. Instalar o Java (versão 8 ou superior) e baixar o arquivo `rars1_6.jar`; abrir com `java -jar rars1_6.jar`.
2. Em Settings, manter habilitadas as pseudo-instruções e a configuração de memória padrão.
3. Abrir `poc/fatorial_completo_tema4.s` e montar (F3). Mostrar a aba *Text Segment*: endereço, código de máquina, instrução básica e fonte — por exemplo, `ble a0, t1, if_caso_base` aparece expandida como um `bge` com os operandos trocados, código `0x00a35c63` (Figura 4).

Demonstração 1 – laço while

4. Executar passo a passo (F7). Na leitura, digitar −3 na janela *Run I/O*: o `bge` em `0x00400018` não é tomado, o PC vai para `0x0040001c`, aparece a mensagem de erro e o `j` em `0x00400030` devolve o PC a `0x00400000`.
5. Digitar 5: o desvio é tomado e o PC salta para `fim_while` (`0x00400034`). Mostrar `a0` = 5 na aba *Registers*.

Demonstração 2 – if/else, recursão e pilha

6. Marcar *breakpoint* em `epilogo` (`0x00400094`) e executar (F5). Na primeira parada, a0 = 1 (caso base).
7. Na janela *Data Segment*, selecionar "current sp" e mostrar os 5 quadros (Figura 5): os `ra` salvos (`0x00400040` e `0x00400084`) e os valores de n de 5 a 1; na aba *Registers*, `sp` = `0x7fffefd4`.
8. Continuar a execução e mostrar `sp` subindo 8 bytes a cada `jr ra` até voltar a `0x7fffeffc`, e `t1` recebendo n antes de cada `mul`.

Demonstração 3 – métricas e estouro

9. Reiniciar, abrir Tools › Instruction Counter, conectar (*Connect to Program*) e executar digitando −3, −1 e 5: o contador deve mostrar 106 instruções (80 se o

valor 5 for digitado direto), uma a menos que o rastreador, porque o RARS não conta o `ecall` de saída.

10. Abrir `poc/fatorial_iterativo.asm`, executar com 5 e comparar no Instruction Counter: 37 instruções contra 80 da versão recursiva, para o mesmo resultado 120.
11. Executar os dois programas digitando 13: ambos imprimem 1932053504; explicar a ausência de *flag* de *overflow*.

Demonstração 4 – falhas

12. Abrir `poc/erros_comuns.asm` com `li s11, 1`: executar e mostrar o PC preso no laço; interromper (Stop).
13. Trocar para `li s11, 2`: mostrar `ra` apontando para o próprio `ret` (`0x00400044`) e o PC preso.
14. Trocar para `li s11, 3`: mostrar a mensagem de exceção de endereço desalinhado.